LAPORAN PRAKTIKUM

Modul 6: Generate Data Otomatis & Authentication



Disusun Oleh:

Nama : Davin Aileenskie Sofiandy

NIM : 707012500135

Kelas : D4SIKC-49-04

Mata Kuliah : Pengembangan Aplikasi Berbasis Web

TELKOM UNIVERSITY

FAKULTAS ILMU TERAPAN

JURUSAN S1 TERAPAN SISTEM INFORMASI KOTA CERDAS

TAHUN AJARAN 2026

1. Tujuan Praktikum

Praktikum Modul 6 ini saya kerjakan untuk menambahkan fitur autentikasi pada project Laravel. Aplikasi yang sebelumnya bisa dibuka oleh siapa saja sekarang perlu memiliki akun dan melewati proses login sebelum halaman tertentu dapat diakses. Ada beberapa tujuan yang ingin saya capai.

1. Memahami cara kerja authentication, session, hashing password, dan middleware auth pada Laravel.
2. Memasang Laravel Breeze dan menjalankan perintah instalasinya pada project Laravel.
3. Menelusuri route, controller, dan view autentikasi yang dibuat Breeze, mulai dari register, login, sampai logout.
4. Menyesuaikan konfigurasi database pada file .env dan menjalankan migration untuk tabel users, password_reset_tokens, dan sessions.
5. Membuat seeder UserSeeder dengan Faker untuk menyiapkan lima akun dummy dan satu akun admin.
6. Menguji proteksi halaman dashboard dan profile dengan middleware auth melalui browser.
7. Memeriksa isi tabel users pada phpMyAdmin untuk memastikan kolom password tersimpan dalam bentuk hash.

2. Alat dan Bahan

Praktikum ini dikerjakan pada laptop pribadi dengan sistem operasi Windows. Seluruh kebutuhan perangkat lunak sudah tersedia dari praktikum sebelumnya, kecuali Laravel Breeze yang baru dipasang pada modul ini.

2.1 Perangkat Lunak

Perangkat Lunak	**Versi / Keterangan**
XAMPP (Apache & MySQL)	Terpasang pada D:\xampp
MySQL / MariaDB	Server database melalui XAMPP, database db_laravel_modul6
phpMyAdmin	Versi 5.2.1, untuk memeriksa tabel users
PHP	8.2.12 (ZTS Visual C++ 2019 x64)
Composer	2.10.3
Laravel Framework	12.70.0
Laravel Breeze	2.4.2
Node.js	25.6.0
NPM	11.8.0
Vite	7.3.7

Visual Studio Code	Editor kode dan terminal project
Git	2.53.0.windows.1
Google Chrome	Untuk menjalankan dan menguji aplikasi

3. Langkah-Langkah Praktikum

Project yang saya pakai bernama PRK_6, yaitu project Laravel baru. Karena project ini belum berisi route dan view dari modul sebelumnya, saya memilih jalur Laravel Breeze seperti pada subbab 6.4.5 sampai 6.4.8. Authentication manual pada subbab 6.4.10 tidak saya kerjakan di project ini karena modul menyarankan pengerjaannya di project tanpa Breeze agar route login dan register tidak bertabrakan. Berikut tahapan yang saya jalankan.

3.1 Instalasi Laravel Breeze

Langkah pertama adalah mengunduh package Breeze. Perintah composer require laravel/breeze --dev menambahkan Breeze sebagai dependency pengembangan pada project. Setelah itu saya menjalankan breeze:install blade untuk membuat route, controller, dan view autentikasi. Pilihan blade berarti tampilan dibuat memakai Blade, bukan React atau Vue. Perintah npm install mengunduh package JavaScript, lalu npm run dev menjalankan Vite untuk meng-compile CSS dan JavaScript sekaligus memantau perubahan berkas.

```
composer require laravel/breeze --dev
php artisan breeze:install blade
npm install
npm run dev
```

Setelah proses instalasi selesai, muncul beberapa berkas baru. Route autentikasi ada di routes/auth.php, view ada di resources/views/auth, dan controller ada di app/Http/Controllers/Auth. Ketiga bagian itu membentuk pola MVC yang sama seperti modul sebelumnya. Bedanya, semuanya sudah dibuat otomatis sehingga saya tinggal memeriksa dan menggunakannya.

3.2 Konfigurasi Database dan Migration

Agar data akun tersimpan, saya membuat database baru bernama db_laravel_modul6 melalui phpMyAdmin, kemudian menyesuaikan bagian database pada file .env. Nilai DB_CONNECTION diisi mysql, DB_HOST 127.0.0.1, DB_PORT 3306, DB_DATABASE db_laravel_modul6, dan

DB_USERNAME root. DB_PASSWORD dibiarkan kosong mengikuti konfigurasi bawaan XAMPP yang tidak memberi password pada user root.

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=db_laravel_modul6
DB_USERNAME=root
DB_PASSWORD=
```

Setelah konfigurasi benar, migration dijalankan untuk membentuk tabel. Perintah berikut membuat tabel users, password_reset_tokens, dan sessions.

```
php artisan migrate
```

Tabel users menyimpan data akun, tabel sessions mencatat sesi login, dan tabel password_reset_tokens dipakai fitur lupa password bawaan Breeze. Project ini juga masih memiliki tabel bawaan Laravel lain seperti cache, cache_locks, jobs, job_batches, dan failed_jobs. Struktur tabel users berisi kolom id, name, email, email_verified_at, password, remember_token, created_at, dan updated_at.

3.3 Route Authentication

Breeze membagi route autentikasi ke dalam dua berkas. routes/web.php hanya berisi route halaman utama, dashboard, dan profil, lalu memuat routes/auth.php di bagian akhir. Route dashboard dilindungi middleware auth dan verified, sedangkan route profil berada dalam grup middleware auth.

```php
<?php

use App\Http\Controllers\ProfileController;
use Illuminate\Support\Facades\Route;

Route::get('/', function () {
    return view('welcome');
});

Route::get('/dashboard', function () {
    return view('dashboard');
})->middleware(['auth', 'verified'])->name('dashboard');

Route::middleware('auth')->group(function () {
    Route::get('/profile', [ProfileController::class, 'edit'])
        ->name('profile.edit');
    Route::patch('/profile', [ProfileController::class, 'update'])
        ->name('profile.update');
    Route::delete('/profile', [ProfileController::class, 'destroy'])
        ->name('profile.destroy');
});

require __DIR__.'/auth.php';
```

Di dalam routes/auth.php terdapat dua grup route. Grup pertama memakai middleware guest, yaitu route yang hanya boleh dibuka oleh pengguna yang belum login seperti register dan login. Grup kedua memakai middleware auth dan berisi route yang membutuhkan login, salah satunya logout. Potongan berikut menampilkan route register, login, dan logout.

```php
<?php

use App\Http\Controllers\Auth\AuthenticatedSessionController;
use App\Http\Controllers\Auth\RegisteredUserController;
use Illuminate\Support\Facades\Route;

Route::middleware('guest')->group(function () {
    Route::get('register', [RegisteredUserController::class, 'create'])
        ->name('register');

    Route::post('register', [RegisteredUserController::class, 'store']);

    Route::get('login', [AuthenticatedSessionController::class, 'create'])
        ->name('login');

    Route::post('login', [AuthenticatedSessionController::class, 'store']);
});

Route::middleware('auth')->group(function () {
    Route::post('logout', [AuthenticatedSessionController::class, 'destroy'])
        ->name('logout');
});
```

Route login diberi nama login. Nama itu wajib ada karena middleware auth memakainya sebagai tujuan pengalihan ketika pengguna yang belum login mencoba membuka halaman terlindungi.

3.4 Controller Authentication

Proses register ditangani RegisteredUserController. Method create menampilkan form register, sedangkan method store memvalidasi input lalu menyimpan akun baru. Validasi memakai unique:users pada email agar tidak ada email ganda, confirmed untuk mencocokkan password dengan password_confirmation, dan Rules\Password::defaults() yang membatasi panjang password minimal delapan karakter. Password disimpan melalui Hash::make($request->password). Setelah akun dibuat, event Registered dipicu, pengguna langsung login lewat Auth::login($user), lalu diarahkan ke dashboard.

```php
<?php

namespace App\Http\Controllers\Auth;

use App\Http\Controllers\Controller;
use App\Models\User;
use Illuminate\Auth\Events\Registered;
use Illuminate\Http\RedirectResponse;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Auth;
use Illuminate\Support\Facades\Hash;
use Illuminate\Validation\Rules;
use Illuminate\Validation\ValidationException;
use Illuminate\View\View;

class RegisteredUserController extends Controller
{
    public function create(): View
    {
        return view('auth.register');
    }

    public function store(Request $request): RedirectResponse
    {
        $request->validate([
            'name' => ['required', 'string', 'max:255'],
            'email' => [
                'required', 'string', 'lowercase', 'email', 'max:255',
                'unique:'.User::class,
            ],
            'password' => ['required', 'confirmed', Rules\Password::defaults()],
        ]);

        $user = User::create([
            'name' => $request->name,
            'email' => $request->email,
            'password' => Hash::make($request->password),
        ]);

        event(new Registered($user));

        Auth::login($user);

        return redirect(route('dashboard', absolute: false));
    }
}
```

Proses login dan logout ditangani AuthenticatedSessionController. Method store memanggil authenticate() dari LoginRequest, membuat ulang ID session dengan regenerate(), lalu mengarahkan pengguna ke halaman yang ingin dibuka atau ke dashboard. Method destroy

menjalankan Auth::guard('web')->logout(), mengosongkan isi session, dan mengganti token CSRF supaya sesi lama tidak bisa dipakai lagi.

```php
<?php

namespace App\Http\Controllers\Auth;

use App\Http\Controllers\Controller;
use App\Http\Requests\Auth\LoginRequest;
use Illuminate\Http\RedirectResponse;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Auth;
use Illuminate\View\View;

class AuthenticatedSessionController extends Controller
{
    public function create(): View
    {
        return view('auth.login');
    }

    public function store(LoginRequest $request): RedirectResponse
    {
        $request->authenticate();

        $request->session()->regenerate();

        return redirect()->intended(route('dashboard', absolute: false));
    }

    public function destroy(Request $request): RedirectResponse
    {
        Auth::guard('web')->logout();

        $request->session()->invalidate();

        $request->session()->regenerateToken();

        return redirect('/');
    }
}
```

Pencocokan email dan password ada di dalam method authenticate() pada LoginRequest. Auth::attempt mencoba mencocokkan password yang diketik dengan hash yang tersimpan di database. Kalau gagal, percobaan dicatat RateLimiter dan pesan These credentials do not match

our records. dikembalikan ke form. Rate limiter membatasi lima percobaan sebelum akun dikunci sementara.

```php
public function authenticate(): void
{
    $this->ensureIsNotRateLimited();

    if (! Auth::attempt(
        $this->only('email', 'password'),
        $this->boolean('remember')
    )) {
        RateLimiter::hit($this->throttleKey());

        throw ValidationException::withMessages([
            'email' => trans('auth.failed'),
        ]);
    }

    RateLimiter::clear($this->throttleKey());
}
```

3.5 View Authentication

View autentikasi Breeze memakai layout guest yang terpisah dari layout aplikasi setelah login. Form register berisi kolom Name, Email, Password, dan Confirm Password, serta tautan ke halaman login. Setiap input memakai komponen x-text-input dan pesan validasi ditampilkan melalui x-input-error.

```blade
<x-guest-layout>
    <form method="POST" action="{{ route('register') }}">
        @csrf

        <div>
            <x-input-label for="name" :value="__('Name')" />
            <x-text-input id="name" class="block mt-1 w-full" type="text"
                name="name" :value="old('name')" required autofocus
                autocomplete="name" />
            <x-input-error :messages="$errors->get('name')" class="mt-2" />
        </div>

        <div class="mt-4">
            <x-input-label for="email" :value="__('Email')" />
            <x-text-input id="email" class="block mt-1 w-full" type="email"
                name="email" :value="old('email')" required
                autocomplete="username" />
            <x-input-error :messages="$errors->get('email')" class="mt-2" />
        </div>

        <div class="mt-4">
            <x-input-label for="password" :value="__('Password')" />
            <x-text-input id="password" class="block mt-1 w-full"
                type="password" name="password" required
                autocomplete="new-password" />
            <x-input-error :messages="$errors->get('password')" class="mt-2" />
        </div>

        <div class="mt-4">
            <x-input-label for="password_confirmation"
                :value="__('Confirm Password')" />
            <x-text-input id="password_confirmation"
                class="block mt-1 w-full" type="password"
                name="password_confirmation" required
                autocomplete="new-password" />
            <x-input-error :messages="$errors->get('password_confirmation')"
                class="mt-2" />
        </div>

        <div class="flex items-center justify-end mt-4">
            <a class="underline text-sm text-gray-600 hover:text-gray-900"
                href="{{ route('login') }}">
                {{ __('Already registered?') }}
            </a>

            <x-primary-button class="ms-4">
                {{ __('Register') }}
            </x-primary-button>
        </div>
    </form>
</x-guest-layout>
```

Form login berisi kolom Email, Password, checkbox Remember me, dan tautan Forgot your password. Data form dikirim ke route login memakai method POST dan dilengkapi @csrf.

```blade
<x-guest-layout>
    <x-auth-session-status class="mb-4" :status="session('status')" />

    <form method="POST" action="{{ route('login') }}">
        @csrf

        <div>
            <x-input-label for="email" :value="__('Email')" />
            <x-text-input id="email" class="block mt-1 w-full" type="email"
                name="email" :value="old('email')" required autofocus
                autocomplete="username" />
            <x-input-error :messages="$errors->get('email')" class="mt-2" />
        </div>

        <div class="mt-4">
            <x-input-label for="password" :value="__('Password')" />
            <x-text-input id="password" class="block mt-1 w-full"
                type="password" name="password" required
                autocomplete="current-password" />
            <x-input-error :messages="$errors->get('password')" class="mt-2" />
        </div>

        <div class="block mt-4">
            <label for="remember_me" class="inline-flex items-center">
                <input id="remember_me" type="checkbox" name="remember"
                    class="rounded border-gray-300 text-indigo-600
                        shadow-sm focus:ring-indigo-500">
                <span class="ms-2 text-sm text-gray-600">
                    {{ __('Remember me') }}
                </span>
            </label>
        </div>

        <div class="flex items-center justify-end mt-4">
            @if (Route::has('password.request'))
                <a class="underline text-sm text-gray-600 hover:text-gray-900"
                    href="{{ route('password.request') }}">
                    {{ __('Forgot your password?') }}
                </a>
            @endif

            <x-primary-button class="ms-3">
                {{ __('Log in') }}
            </x-primary-button>
        </div>
    </form>
</x-guest-layout>
```

Halaman yang muncul setelah login adalah dashboard. Isinya masih sederhana, yaitu pesan You're logged in! sebagai penanda bahwa proses login berhasil.

```blade
<x-app-layout>
    <x-slot name="header">
        <h2 class="font-semibold text-xl text-gray-800 leading-tight">
            {{ __('Dashboard') }}
        </h2>
    </x-slot>

    <div class="py-12">
        <div class="max-w-7xl mx-auto sm:px-6 lg:px-8">
            <div class="bg-white overflow-hidden shadow-sm sm:rounded-lg">
                <div class="p-6 text-gray-900">
                    {{ __("You're logged in!") }}
                </div>
            </div>
        </div>
    </div>
</x-app-layout>
```

Selain dashboard, Breeze menyediakan halaman profile pada resources/views/profile/edit.blade.php. Halaman itu memuat tiga bagian, yaitu Profile Information untuk mengubah nama dan email, Update Password untuk mengganti password, dan Delete Account untuk menghapus akun.

3.6 Navigasi dengan Directive @auth

Menu navigasi berada pada resources/views/layouts/navigation.blade.php. Bagian kanan navbar memakai directive @auth untuk menampilkan nama pengguna beserta menu Profile dan Log Out. Tombol Log Out dikirim melalui form POST dan dilengkapi @csrf karena route logout memang memakai method POST. Ketika pengguna belum login, halaman tamu tetap menyediakan tautan Log in dan Register.

3.7 Membuat Seeder Akun Dummy dengan Faker

Pendaftaran akun satu per satu melalui form terlalu lama untuk pengujian, apalagi setiap kali ingin mencoba login. Saya membuat UserSeeder dengan perintah php artisan make:seeder UserSeeder, lalu mengisinya memakai Faker. Faker::create() tanpa locale menghasilkan nama berbahasa Inggris. Perulangan for ($i = 0; $i < 5; $i++) membuat lima akun dummy, dan email diisi dengan $faker->unique()->safeEmail agar tidak ada email kembar. Kelima akun dummy memakai password yang sama, yaitu password, sedangkan satu akun admin memakai email admin@example.com dan password admin123. Kedua nilai itu tetap disimpan melalui Hash::make supaya kolom password berisi hash, bukan teks asli.

```php
<?php

namespace Database\Seeders;

use Faker\Factory as Faker;
use Illuminate\Database\Seeder;
use Illuminate\Support\Facades\DB;
use Illuminate\Support\Facades\Hash;

class UserSeeder extends Seeder
{
    public function run(): void
    {
        $faker = Faker::create();

        for ($i = 0; $i < 5; $i++) {
            DB::table('users')->insert([
                'name' => $faker->name,
                'email' => $faker->unique()->safeEmail,
                'password' => Hash::make('password'),
                'created_at' => now(),
                'updated_at' => now(),
            ]);
        }

        DB::table('users')->insert([
            'name' => 'Admin Demo',
            'email' => 'admin@example.com',
            'password' => Hash::make('admin123'),
            'created_at' => now(),
            'updated_at' => now(),
        ]);
    }
}
```

Seeder dijalankan memakai opsi --class=UserSeeder. File DatabaseSeeder bawaan tidak saya ubah karena modul menjalankan seeder tersebut secara khusus.

3.8 Menjalankan Seeder dan Menguji Aplikasi

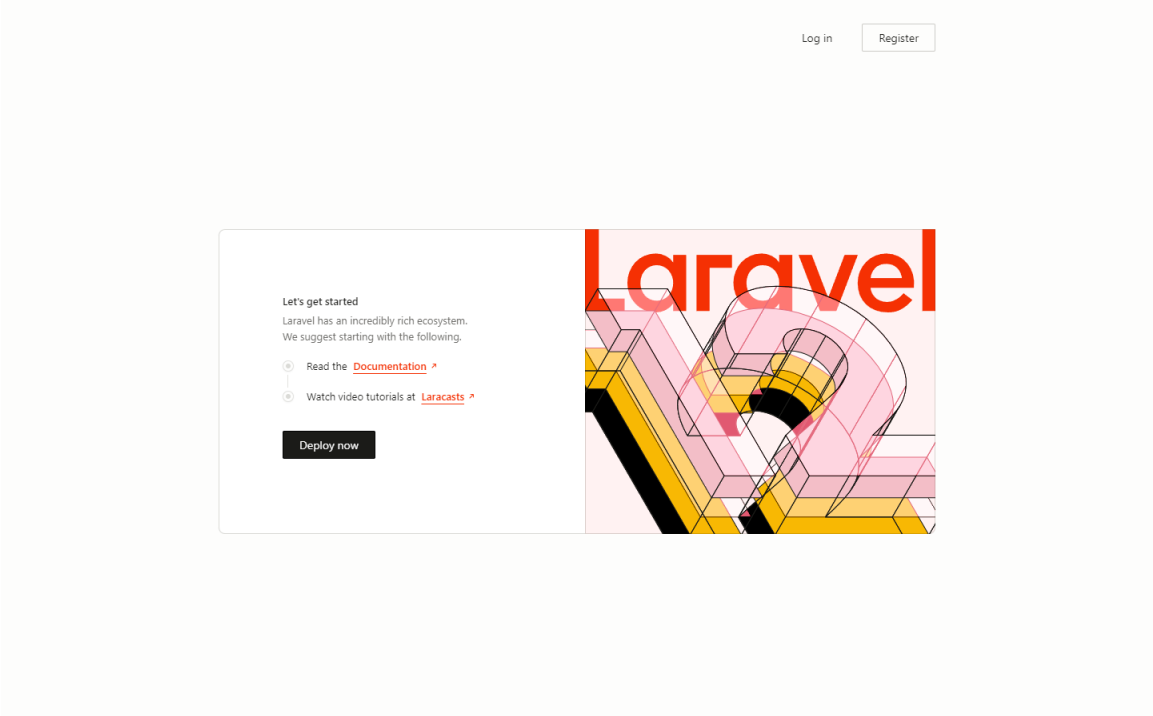
Setelah migration dan seeder siap, saya menjalankan seeder lalu menyalakan server pengembangan pada alamat http://127.0.0.1:8000. Terminal npm run dev dari langkah 3.1 tetap dibiarkan berjalan selama pengujian.

```
php artisan db:seed --class=UserSeeder
php artisan serve
```

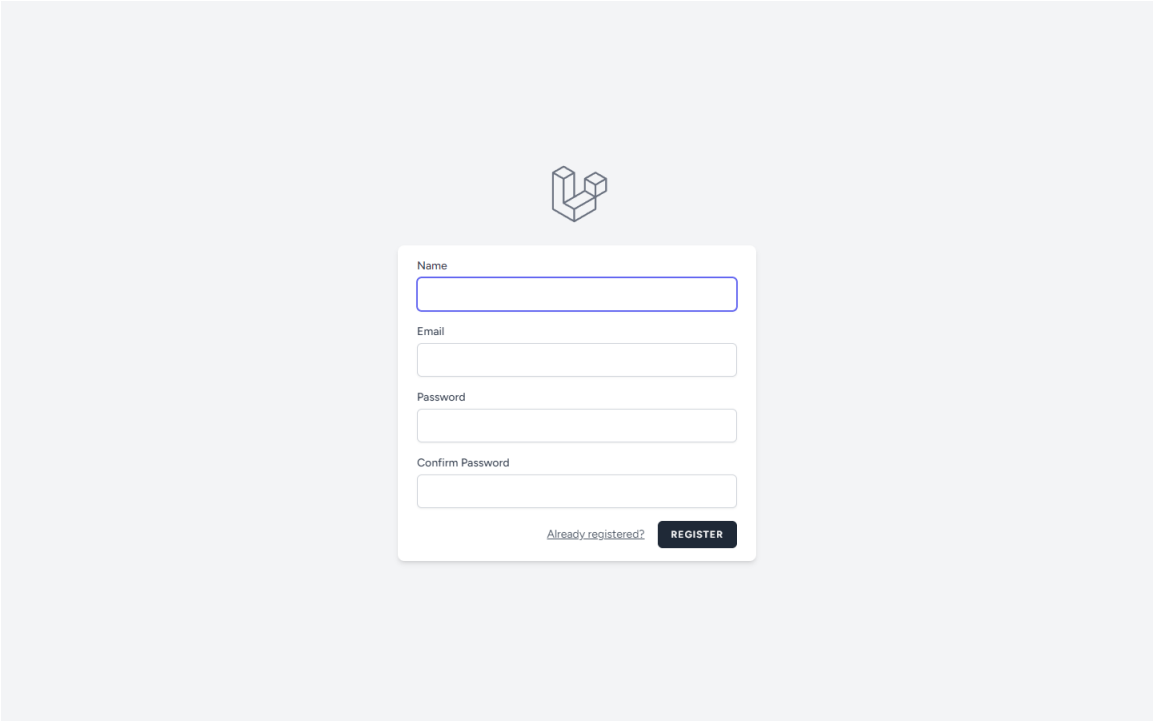
Hasil seeder diperiksa melalui phpMyAdmin. Tabel users bertambah enam baris, yaitu lima akun dummy dan satu akun admin. Sebelumnya sudah ada satu akun hasil pendaftaran melalui form, yaitu davin.aileenskie1@gmail.com, sehingga jumlah baris seluruhnya menjadi tujuh. Pengujian di browser saya lakukan dengan akun admin@example.com dan password admin123. Urutan pengujiannya adalah membuka /dashboard sebelum login, mengisi form register, mencoba login dengan password salah, login memakai akun admin, membuka halaman profile, dan logout.

4. Hasil dan Dokumentasi Program

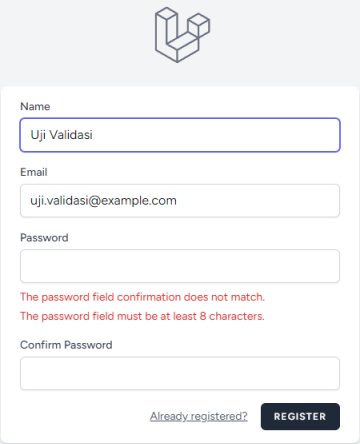
Seluruh alur authentication berjalan sesuai harapan, mulai dari pembatasan halaman, pendaftaran akun, login, sampai logout. Berikut dokumentasi hasil pengujian.



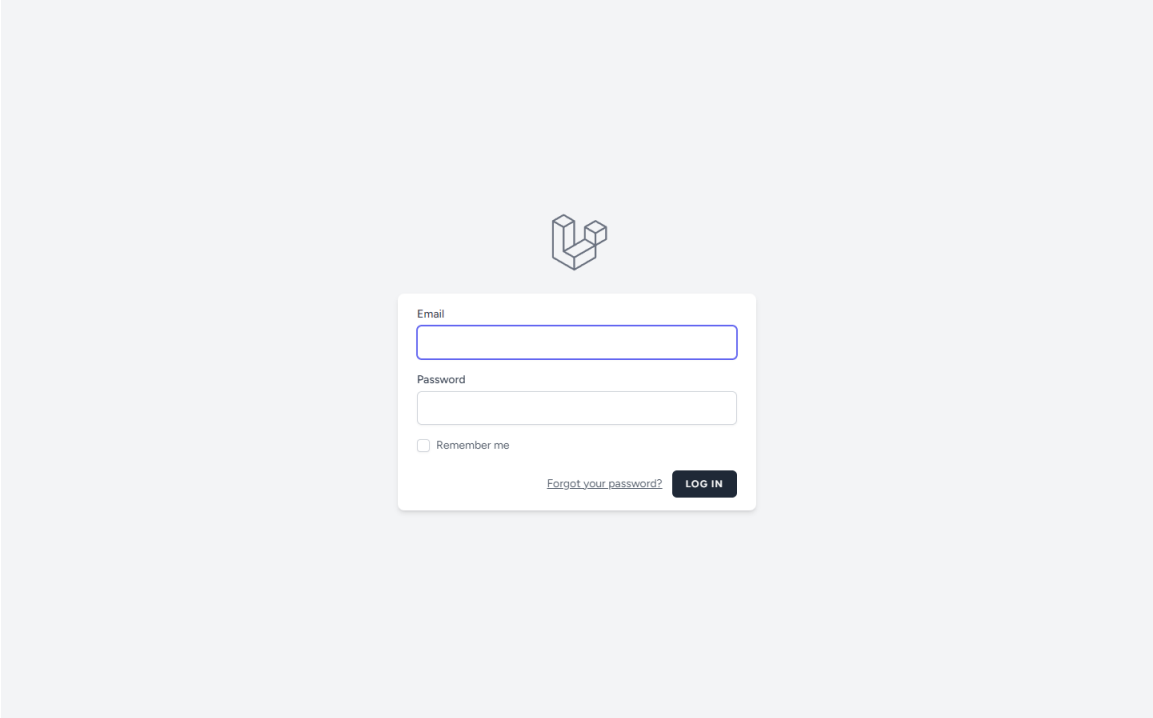
Gambar 1 menampilkan halaman awal aplikasi. Di pojok kanan atas ada tautan Log in dan Register, sedangkan tombol Dashboard baru muncul setelah pengguna masuk.



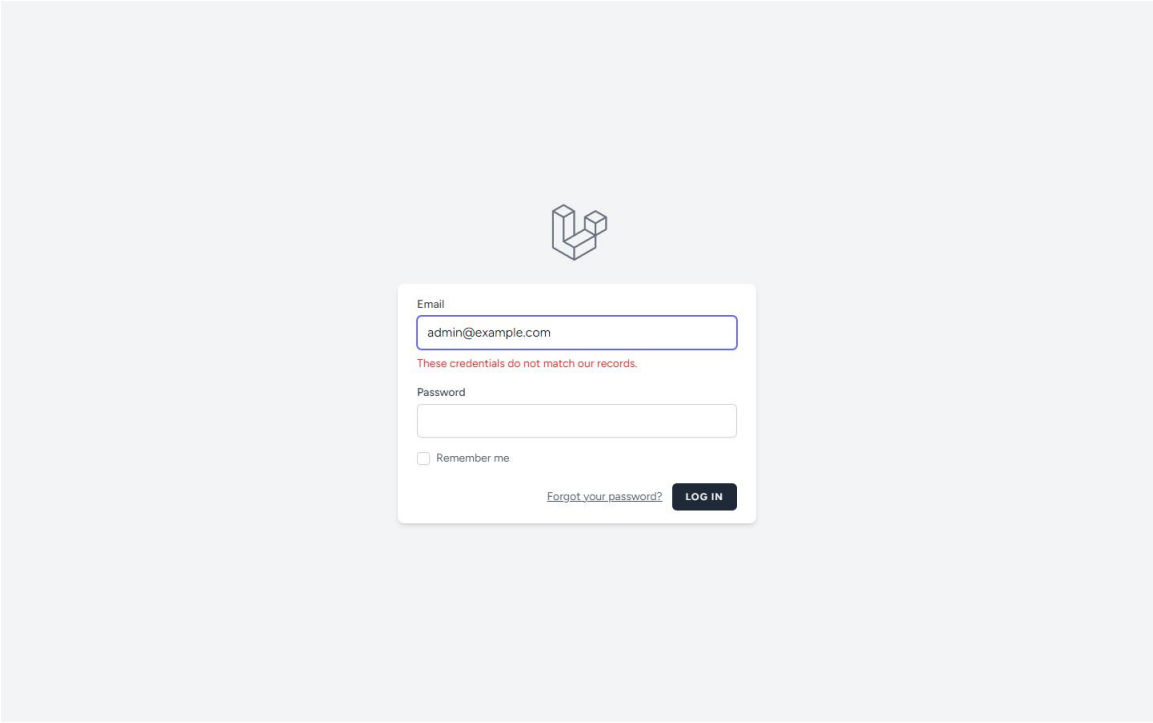
Gambar 2 menampilkan halaman register dengan kolom Name, Email, Password, dan Confirm Password. Form mengirim data ke route register memakai method POST dan sudah dilengkapi @csrf.



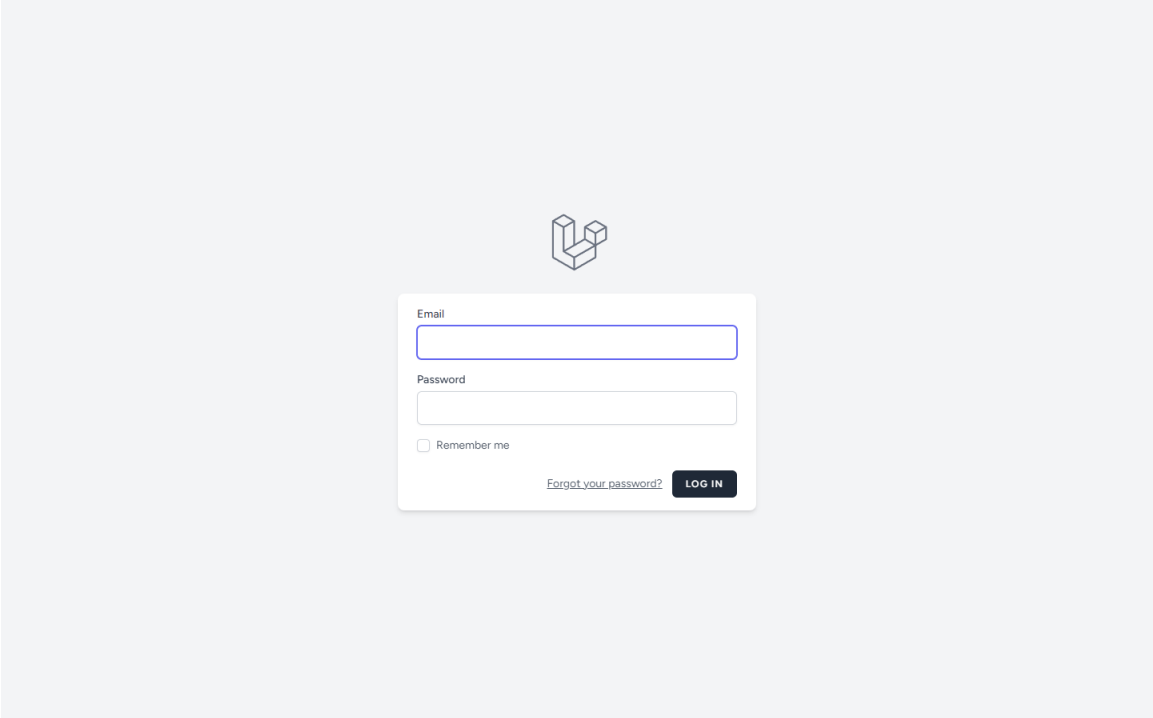
Gambar 3 menampilkan pesan validasi ketika password diisi 123 dan konfirmasinya diisi 456. Muncul dua pesan sekaligus, yaitu The password field confirmation does not match. dan The password field must be at least 8 characters. Kedua pesan itu berasal dari aturan confirmed dan Rules\Password::defaults(). Kolom Name dan Email masih terisi data sebelumnya karena view memakai old(), sedangkan kolom password selalu dikosongkan lagi.



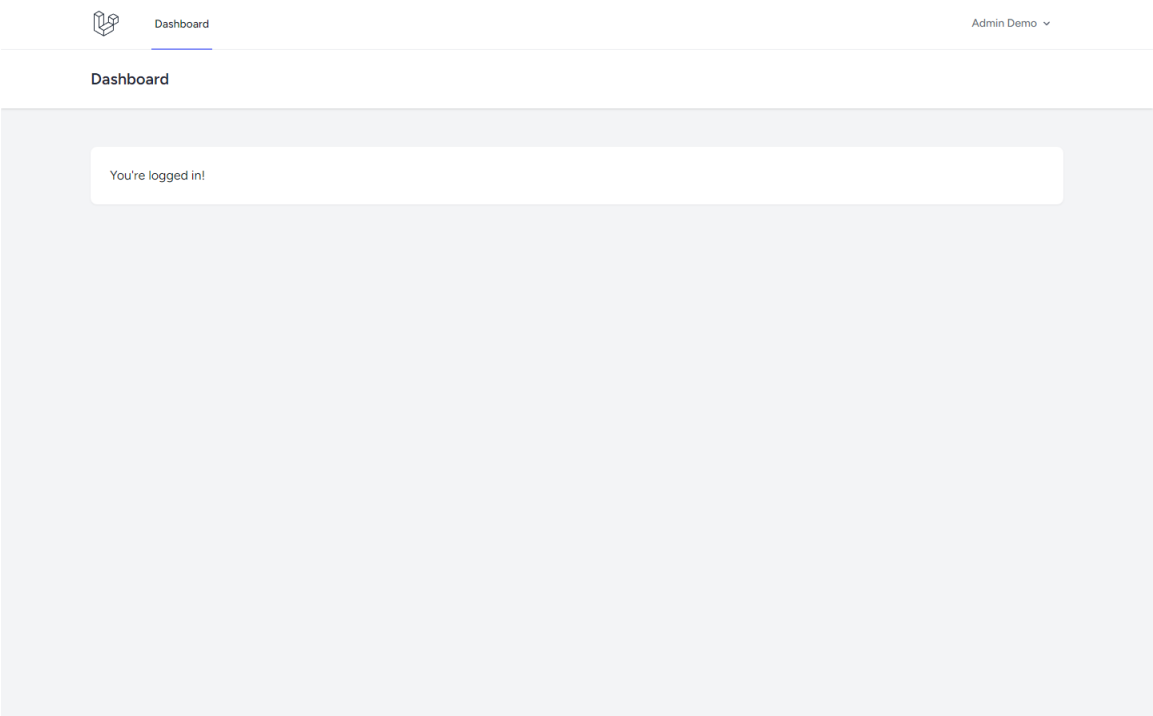
Gambar 4 menampilkan halaman login dengan kolom Email, Password, checkbox Remember me, dan tautan Forgot your password.



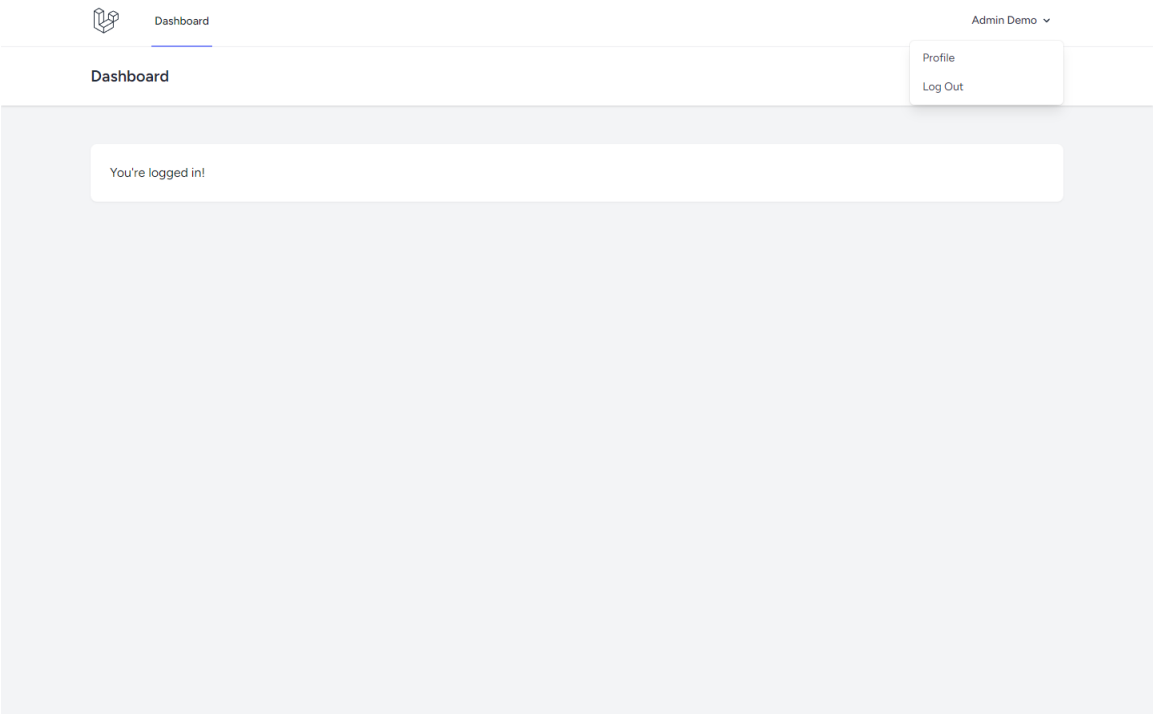
Gambar 5 menampilkan pesan These credentials do not match our records. Ketika saya sengaja memasukkan password yang salah untuk akun admin@example.com, proses login ditolak dan saya tetap berada di halaman login.



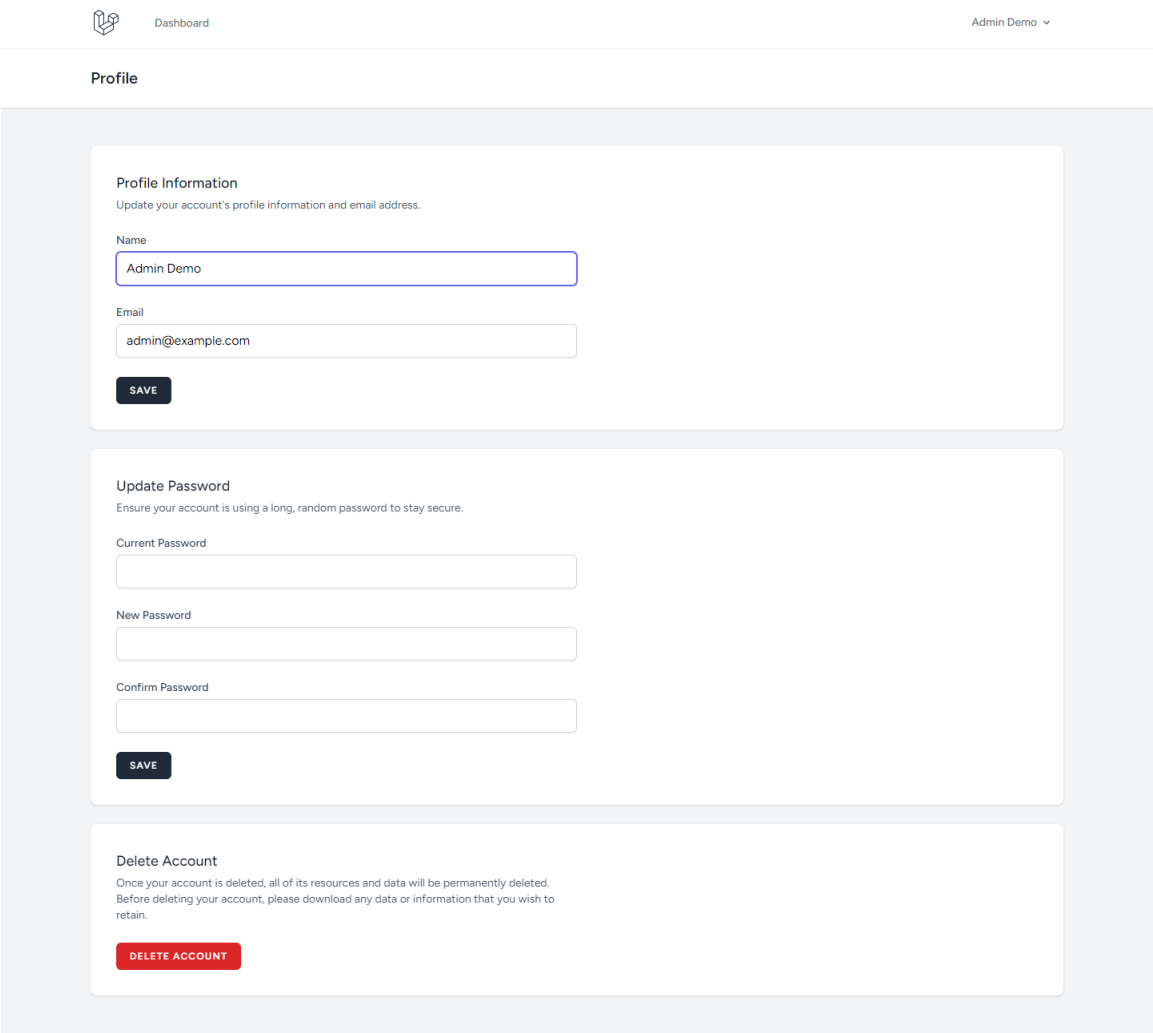
Gambar 6 menampilkan halaman login yang muncul ketika saya membuka http://127.0.0.1:8000/dashboard tanpa login terlebih dahulu. Alamat pada browser berubah menjadi /login, tanda middleware auth menahan permintaan ke dashboard dan mengarahkan pengguna ke route login.



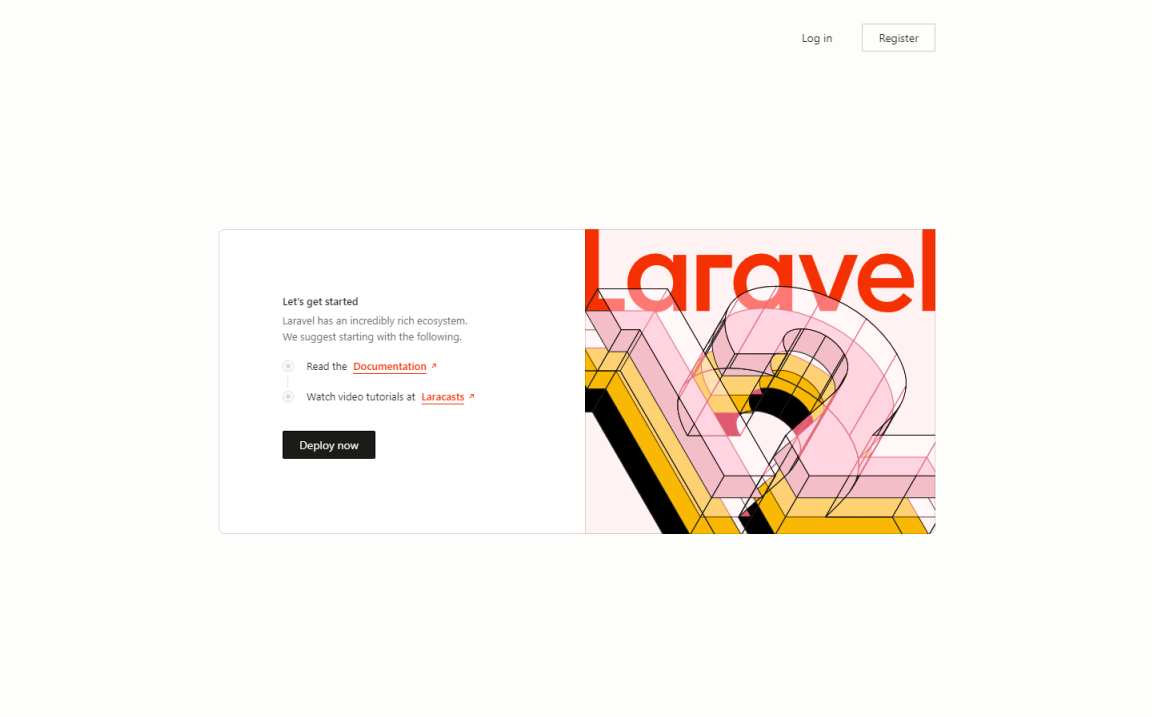
Gambar 7 menampilkan dashboard setelah login memakai akun admin@example.com. Nama Admin Demo terlihat di pojok kanan atas dan pesan You're logged in! menandakan sesi login aktif.



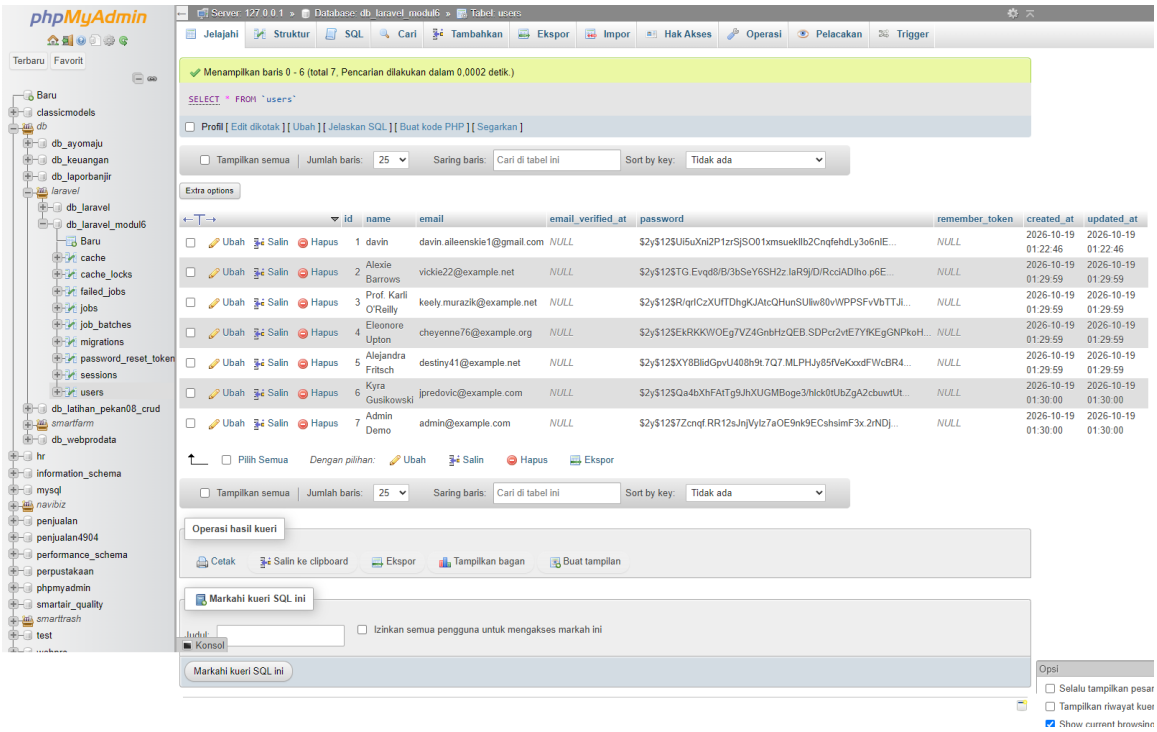
Gambar 8 menampilkan menu dropdown di kanan atas yang berisi Profile dan Log Out. Menu ini muncul karena directive @auth pada layout navigasi.



Gambar 9 menampilkan halaman profile. Bagian Profile Information berisi nama Admin Demo dan email admin@example.com, di bawahnya ada bagian Update Password dan Delete Account.



Gambar 10 menampilkan halaman welcome setelah proses logout. Tautan Log in dan Register kembali muncul, tanda sesi login sudah dihapus.



Gambar 11 menampilkan tabel users pada phpMyAdmin. Terdapat tujuh baris data. Kolom password berisi deretan karakter yang diawali $2y$12$, yaitu hasil hashing bcrypt, bukan password asli. Baris pertama berasal dari akun yang didaftarkan melalui form, lima baris berikutnya berasal dari Faker, dan baris terakhir adalah akun admin@example.com.

5. Analisis dan Pembahasan

1. Perbedaan aplikasi sebelum dan sesudah menggunakan authentication.

Project Laravel baru bisa dibuka oleh siapa saja, termasuk halaman dashboard yang sebenarnya hanya berguna untuk pengguna yang sudah punya akun. Setelah Breeze dipasang, permintaan ke dashboard dan profile melewati middleware auth lebih dulu. Tamu yang mencoba membuka keduanya diarahkan ke halaman login, sedangkan halaman welcome, login, dan register tetap bisa diakses supaya calon pengguna masih dapat mendaftar. Perubahan ini membuat aplikasi punya batas jelas antara area publik dan area yang butuh akun.

2. Perbedaan authentication dengan Breeze dan secara manual.

Jalur Breeze lebih cepat karena route, controller, dan view dibuat sekaligus oleh perintah artisan. Kekurangannya, kode yang dihasilkan cukup banyak sehingga alur di balik layar tidak langsung terlihat dan perlu dibaca satu per satu. Jalur manual pada subbab 6.4.10 menuntut RegisterController dan LoginController ditulis sendiri, termasuk validasi, Hash::make(), Auth::attempt(), dan Auth::logout(). Cara itu lebih jelas untuk belajar, tetapi harus dikerjakan pada project tanpa Breeze karena route login dan register dari keduanya bisa bertabrakan. Karena PRK_6 sudah memakai Breeze, saya tidak menggabungkan kedua pendekatan itu dalam satu project.

3. Peran session, hashing password, dan middleware auth.

Session dipakai Laravel untuk mengingat bahwa seseorang sudah login. Datanya tersimpan di server dan hanya penanda sesinya yang dikirim ke browser melalui cookie, sehingga pengguna tidak perlu mengetik ulang email dan password setiap berpindah halaman. Kolom password pada tabel users tidak pernah menyimpan teks asli. Nilai yang tampak seperti $2y$12$ di phpMyAdmin adalah hasil bcrypt, dan Laravel mencocokkannya saat login memakai Hash::check() di dalam Auth::attempt. Middleware auth menjadi penyaring sebelum request sampai ke controller. Kalau session tidak ditemukan, request tidak diteruskan ke halaman yang dilindungi dan pengguna diarahkan ke route login.

4. Alasan pemilihan halaman yang wajib login.

Dashboard dan profile saya biarkan di balik middleware auth karena keduanya menampilkan informasi milik pengguna yang sedang login. Halaman welcome, login, dan register sengaja tetap terbuka. Halaman login dan register memang harus bisa dijangkau oleh pengguna yang belum punya sesi, sedangkan welcome berfungsi sebagai halaman awal aplikasi.

5. Kendala dan penyelesaian.

Selama pengujian di project ini tidak ada galat yang tercatat pada storage/logs/laravel.log. Dua pesan yang muncul justru berasal dari validasi yang memang dirancang, yaitu pesan konfirmasi password pada Gambar 3 dan pesan login gagal pada Gambar 5. Keduanya menandakan validasi dan pemeriksaan kredensial bekerja. Modul menyebut beberapa galat yang sering muncul pada pengerjaan authentication. Berikut catatan saya untuk kasus tersebut, meskipun tidak semuanya saya alami langsung.

1. Galat 419 Page Expired muncul ketika token CSRF tidak ikut terkirim atau sesi sudah kedaluwarsa. Penanganannya adalah memastikan setiap form memiliki @csrf dan konfigurasi session berjalan normal. Form Breeze sudah memuat @csrf, jadi saya tidak mengubahnya.

2. Galat Route [login] not defined terjadi ketika middleware auth mengarahkan pengguna ke route bernama login, tetapi route itu belum diberi nama. Pada routes/auth.php nama login sudah ada, sehingga pengalihan berjalan.

3. Login selalu gagal biasanya karena password di database disimpan sebagai teks biasa. Laravel mencocokkan password dengan nilai hash, jadi akun hasil seeder harus dibuat memakai Hash::make seperti pada UserSeeder.

4. Galat Duplicate entry admin@example.com muncul kalau UserSeeder dijalankan lebih dari sekali karena kolom email bersifat unik. Solusinya menghapus baris admin tersebut atau menjalankan php artisan migrate:fresh lalu seed ulang.

5. Perintah npm tidak dapat dijalankan di PowerShell karena eksekusi script diblokir. Perintah Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass membuka izin untuk jendela terminal yang sedang dipakai.

6. Galat Vite manifest not found muncul ketika npm run dev belum dijalankan atau sudah berhenti. Selama pengujian, terminal npm run dev saya biarkan tetap terbuka.

6. Kesimpulan

Berdasarkan praktikum yang sudah saya kerjakan, fitur authentication berhasil dipasang pada project Laravel memakai Laravel Breeze. Perintah breeze:install blade menyediakan route, controller, dan view untuk register, login, dan logout, sedangkan migration membentuk tabel users, password_reset_tokens, dan sessions di database db_laravel_modul6. Password akun disimpan dalam bentuk hash bcrypt, bukan teks asli, dan proses login mencocokkannya melalui Auth::attempt. Middleware auth terbukti menahan akses ke halaman dashboard sebelum login dan mengarahkan pengguna ke halaman login. Seeder UserSeeder dengan Faker berhasil membuat lima akun dummy dan satu akun admin yang bisa dipakai untuk login, sehingga pengujian tidak perlu dilakukan dengan mendaftar akun satu per satu. Keseluruhan alur dari register, login, proteksi halaman, sampai logout berjalan sesuai tujuan praktikum.

7. Link GitHub

Kode program lengkap praktikum ini dapat diakses pada repositori berikut.

https://github.com/davinaileenskie1-hue/PABW-Semester-3